

PROJECT REPORT

MultiMethod DiffSolver

A MATLAB App Designer Based Differential Equation Solving System

Subject:	Numerical Computing and Computer Applications
Project Title:	MultiMethod DiffSolver
Name	Jawaria Tariq (AI Engineer)
Tool Used:	MATLAB (App Designer, Symbolic Math Toolbox)

Table of Contents

1. Introduction.....	3
2. Objectives	3
3. Tools and Technologies Used	3
4. System Design and GUI Layout.....	4
5. Methodology	4
6. Code Structure and Key Functions	5
7. Results and Output.....	6
8. Comparative Discussion	6
9. Conclusion	7
10. References.....	7

1. Introduction

Differential equations form the mathematical backbone of countless engineering and scientific systems, describing everything from population growth and heat transfer to electrical circuits and mechanical vibrations. Solving these equations — whether analytically or numerically — is a core skill in numerical computing, yet the process of switching between solution techniques, verifying results, and visualizing behaviour is often fragmented across separate tools and scripts.

MultiMethod DiffSolver addresses this gap by providing a single, interactive MATLAB application that allows a user to enter a differential equation once and solve it using three distinct approaches: exact symbolic solving, the Euler method, and the fourth-order Runge-Kutta (RK4) method. Built entirely with MATLAB App Designer, the tool combines a clean graphical interface, real-time plotting, and exportable PDF reporting into one cohesive workflow, making it a practical companion for coursework in Numerical Computing and Computer Applications.

2. Objectives

- Design a single, unified MATLAB GUI application capable of solving ordinary and symbolic differential equations.
- Implement and compare two classical numerical integration techniques: the Euler method and the fourth-order Runge-Kutta (RK4) method.
- Integrate MATLAB's Symbolic Math Toolbox to provide exact, closed-form solutions where possible.
- Provide real-time graphical visualization of the solution curve alongside the numerical/symbolic output.
- Allow the user to export the generated solution and plot as a standalone PDF report.
- Build an intuitive, guided interface with built-in usage examples so that a first-time user can operate the tool without external documentation.

3. Tools and Technologies Used

The project is implemented entirely within the MATLAB environment, combining App Designer for the interface with the Symbolic Math Toolbox for exact equation solving. The table below summarizes the core components.

Component	Description
Platform	MATLAB (App Designer)
Toolbox	Symbolic Math Toolbox (dsolve, str2sym, matlabFunction)
Interface	uifigure-based GUI: panels, dropdowns, buttons, text areas, and UIAxes
Numerical Methods	Euler Method, 4th-Order Runge-Kutta (RK4)
Output	On-screen solution list, live plot, exportable PDF report

Component	Description
Language	MATLAB Object-Oriented App (classdef, matlab.apps.AppBase)

4. System Design and GUI Layout

The application is structured as a MATLAB class (main) that inherits from `matlab.apps.AppBase`, following the standard App Designer architecture. The interface is divided into three functional regions:

- Header Panel — displays the application title, "MultiMethod DiffSolver", on a dark navy background for visual branding.
- Input & Control Panel — contains the equation entry field, a solving-method dropdown (Differential Equation, Ordinary Differential Equation, RK4), and four action buttons: Solve, Export PDF, Help, and Clear.
- Output Panel — a text area listing the computed (x, y) values or the symbolic expression, paired with a UIAxes graph that plots the solution curve.

This layout keeps every control visible on a single screen (700 × 480 px window), minimizing the number of steps a user needs to go from entering an equation to viewing a solved, plotted result.

5. Methodology

Three independent solving pathways are implemented behind a single "Solve" action, selected through the method dropdown.

5.1 Symbolic Solution (Differential Equation Mode)

When the user selects "Differential Equation", the entered expression is normalized — converting shorthand such as dy/dx or $\text{diff}(y,x)$ into the symbolic form $\text{diff}(y(x),x)$, and a bare $=$ into $==$. The cleaned expression is parsed with `str2sym` and solved exactly using MATLAB's `dsolve` function, which returns a closed-form expression for $y(x)$, including arbitrary constants (C1, C2) where the equation is not fully determined. For visualization, the constants are substituted with a default value of 1 and the resulting curve is plotted with `fplot` over the interval $[0, 5]$.

5.2 Euler Method (Ordinary Differential Equation Mode)

The Euler method is the simplest first-order numerical technique for approximating the solution of an initial value problem $dy/dx = f(x, y)$. Starting from an initial point (x_0, y_0) , it advances the solution step-by-step using a fixed step size h :

$$y(i+1) = y(i) + h \cdot f(x(i), y(i))$$

In the application, this is implemented in the `odeEulerSolver` helper function, which runs over 50 steps between $x = 0$ and $x = 5$, starting from the initial condition $y(0) = 1$.

5.3 Fourth-Order Runge-Kutta Method (RK4 Mode)

The RK4 method improves on Euler's approximation by sampling the slope function $f(x, y)$ at four points within each step and combining them in a weighted average, giving a much smaller local truncation error. The `rk4Solver` function computes, at every step:

- $k_1 = f(x, y)$
- $k_2 = f(x + h/2, y + h \cdot k_1/2)$
- $k_3 = f(x + h/2, y + h \cdot k_2/2)$
- $k_4 = f(x + h, y + h \cdot k_3)$

and updates the solution using $y(i+1) = y(i) + (h/6)(k_1 + 2k_2 + 2k_3 + k_4)$, producing a smooth, highly accurate curve — as reflected in the sample output shown in Section 7, where the curve rises to a peak near $x \approx 1.4$ before decaying smoothly toward $x = 5$.

6. Code Structure and Key Functions

The application logic is organized into three groups of methods within the main classdef, following App Designer conventions:

6.1 Numerical Solvers

- `rk4Solver(f, x0, y0, xEnd, steps)` — implements the fourth-order Runge-Kutta algorithm described in Section 5.3.
- `odeEulerSolver(f, x0, y0, xEnd, steps)` — implements the first-order Euler method described in Section 5.2.

6.2 Callback Functions

- `btnSolveButtonPushed` — reads the equation and selected method, dispatches to the correct solver (symbolic or numeric), formats the output text, and updates the plot. All logic is wrapped in a try/catch block so malformed input produces a readable error message instead of crashing the app.
- `btnHelpButtonPushed` — opens a uialert dialog listing example equations for each of the three solving modes, acting as inline documentation.
- `btnClearButtonPushed` — resets the equation field to its default example, clears the solution text area, and clears the plot axes.
- `btnPDFButtonPushed` — opens a Save dialog (`uiputfile`), renders the equation, method, and solution text onto an invisible figure, and exports it as a vector PDF using `exportgraphics`.

6.3 Component Initialization

The `createComponents` method builds every visual element — the header panel, labels, edit field, dropdown, buttons, text area, and axes — and assigns their positions, colors, and fonts, giving the interface its consistent dark-navy-and-white theme.

6.4 Representative Code Snippet — RK4 Solver

```
function [x, y] = rk4Solver(~, f, x0, y0, xEnd, steps)
    h = (xEnd - x0) / steps;
    x = zeros(steps + 1, 1);
    y = zeros(steps + 1, 1);
```

```

x(1) = x0; y(1) = y0;
for i = 1:steps
    xi = x(i); yi = y(i);
    k1 = f(xi, yi);
    k2 = f(xi + h/2, yi + h*k1/2);
    k3 = f(xi + h/2, yi + h*k2/2);
    k4 = f(xi + h, yi + h*k3);
    y(i+1) = yi + (h/6)*(k1 + 2*k2 + 2*k3 + k4);
    x(i+1) = xi + h;
end
end

```

7. Results and Output

The figure below shows the application in operation, solving the equation $\sin(x \cdot y) + \exp(-x)$ using the RK4 method with initial condition $y(0) = 1$ over the interval $x \in [0, 5]$. The solution list on the left displays the computed (x, y) pairs at each step, while the panel on the right plots the resulting curve, which rises to a peak of approximately $y = 2.46$ near $x = 1.4$ before decreasing smoothly.

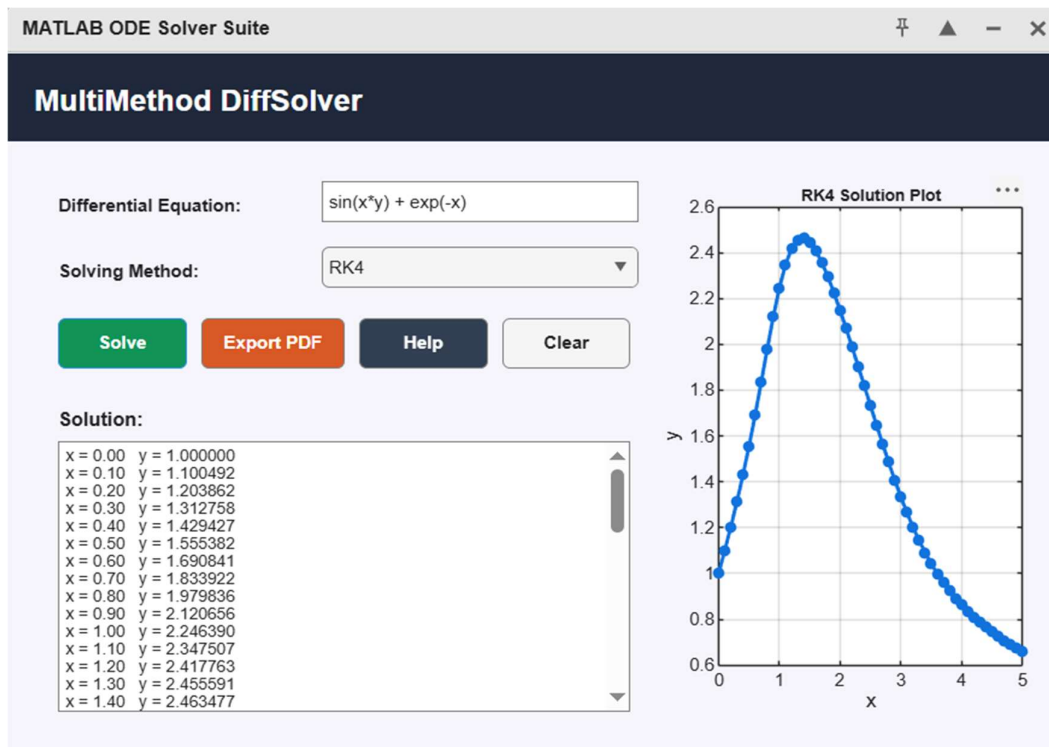


Figure 1: MultiMethod DiffSolver — RK4 solution of $\sin(x \cdot y) + \exp(-x)$

This behaviour is consistent with expectations for the RK4 method: because it evaluates the slope function at four points per step and combines them in a weighted average, the resulting curve is noticeably smoother and more accurate than an equivalent Euler-method run at the same step size, with no visible oscillation or drift over the 50-step interval.

8. Comparative Discussion

The application's ability to switch between solving modes on the same equation makes the trade-offs between numerical methods directly observable. The table below summarizes the key differences between the Euler and RK4 methods as implemented in this project.

Criteria	Euler Method	RK4 Method
Order of Accuracy	First-order ($O(h)$)	Fourth-order ($O(h^4)$)
Function Evaluations / Step	1	4
Computational Cost	Low	Moderate
Stability / Smoothness	Prone to drift on stiff or oscillatory equations	Stable and smooth for a much wider range of equations
Best Suited For	Quick approximations, teaching basic concepts	Accurate simulations where precision matters

Where a closed-form solution exists, the symbolic dsolve mode provides an exact benchmark against which both numerical methods can be validated, closing the loop between analytical and computational approaches within a single tool.

9. Conclusion

MultiMethod DiffSolver successfully demonstrates how symbolic computation and numerical integration techniques can be unified within a single, user-friendly MATLAB application. By combining the Euler method, the fourth-order Runge-Kutta method, and exact symbolic solving under one interface, the project allows direct, side-by-side exploration of how different solution strategies behave on the same differential equation.

Beyond its numerical core, the inclusion of input validation, a built-in help guide, and one-click PDF export reflects sound software design practice, turning a course exercise in numerical methods into a complete, usable tool. Future extensions could include additional methods such as the Midpoint or Adams-Bashforth schemes, adjustable step size and interval controls, and support for systems of coupled differential equations.

10. References

- MathWorks. "App Designer Documentation." MATLAB Documentation.
- MathWorks. "Symbolic Math Toolbox — dsolve." MATLAB Documentation.
- Chapra, S. C., & Canale, R. P. "Numerical Methods for Engineers." McGraw-Hill Education.
- Course Material — Numerical Computing and Computer Applications, Department of AI, QUEST Nawabshah.